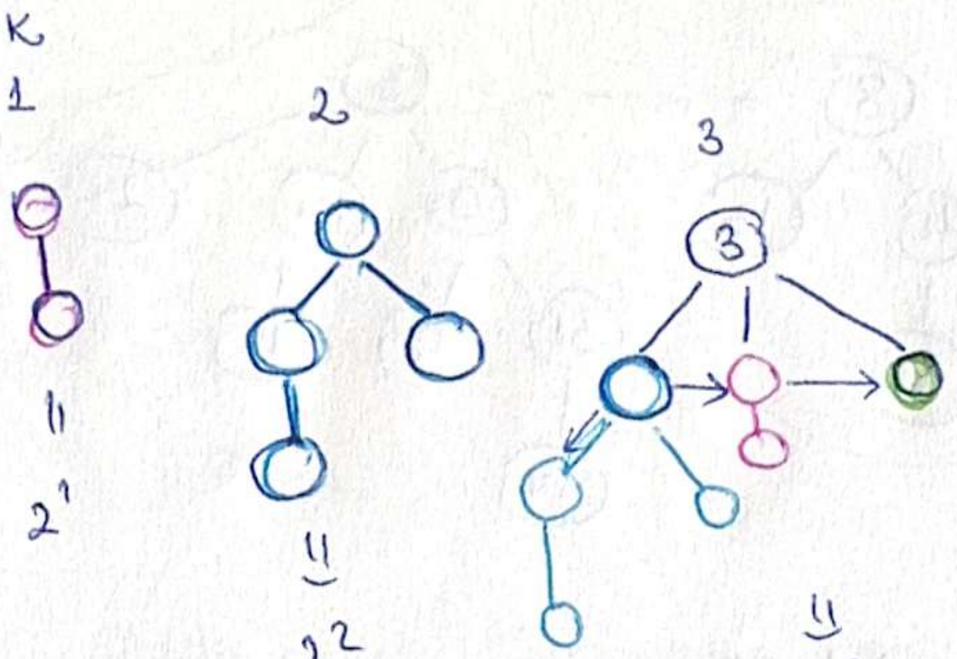
Lecture 9 - DSA

29.04.2026

1) Binomial heap = collection of binomial trees.

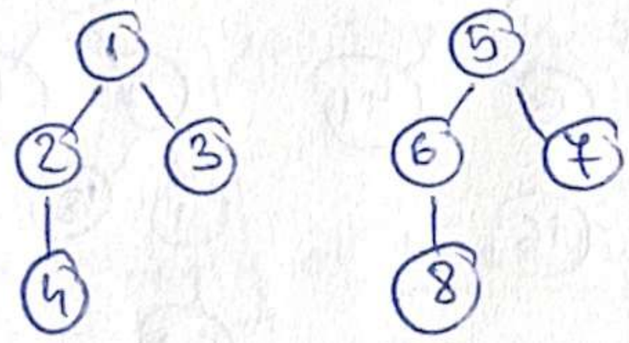
BT: order 0
binomial tree
binary tree ?

Nr. nodes = pw of 2

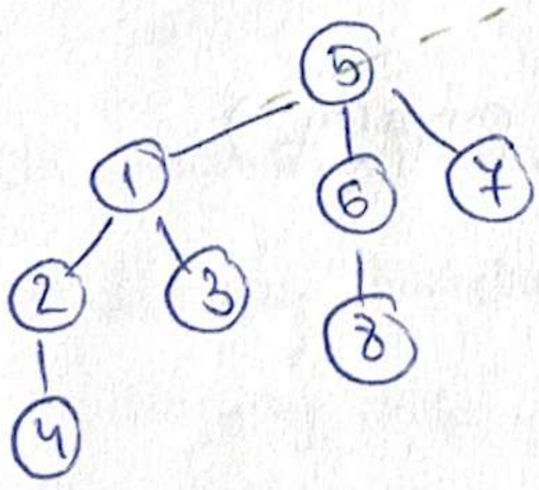


7 nodes: 4+2+1

2 bt of order 2. how to merge them? in $\Theta(1)$ time.



=> they respect the min-heap property.
=> make (5) the leftmost child of (1)
=> the property is respected.



Remove the root => valid BT.

32+
16
48

2) BT implementation

- Keep adr. of root node
- each node holds adr. of the one on its left/right below

How many BT inside a BH?

→ Look at powers.

7 nodes = $4 + 2 + 1$ ($2^2\ 2^1\ 2^0$) => 3 BT.

122 nodes = 1 + 2 + 4 + 8 + 16 + 25 + ...

(SUM OF POWERS OF 2) => ...

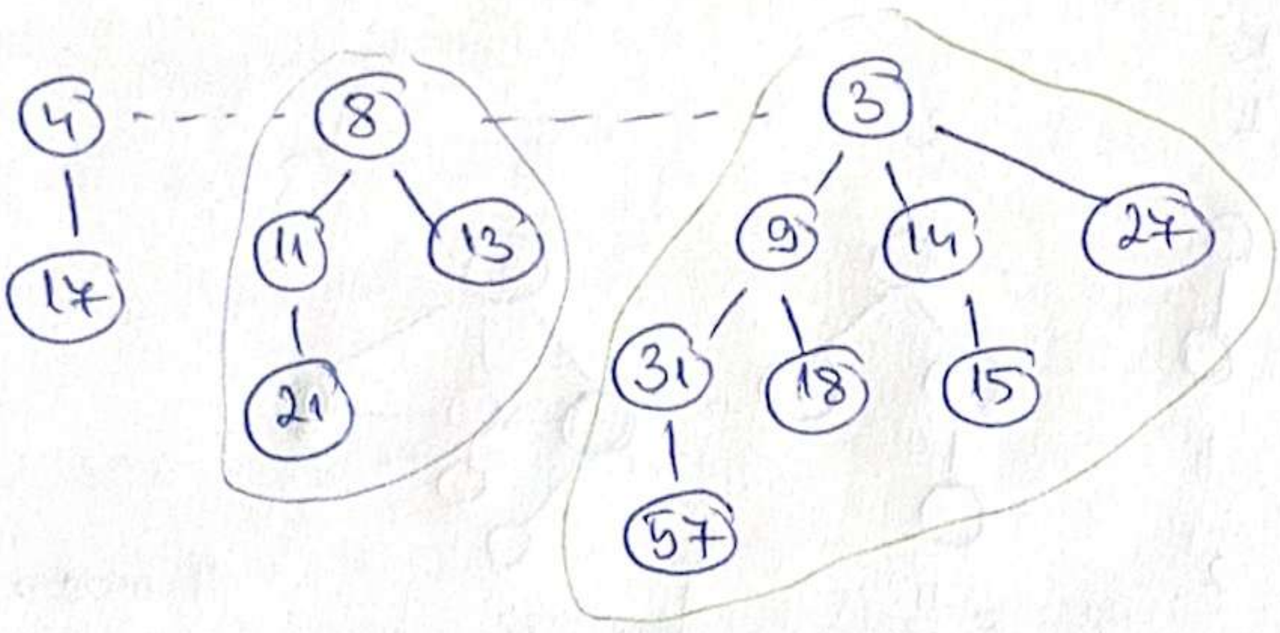
Quiz 9: 37 nodes = $2^5 + 2^2 + 2^0$
$= 32 + 4 + 1$
$= 37$. ⇓ 3 trees.

$84 = 2^6$
$52 = 2^5 + 2^4 + 2^2$

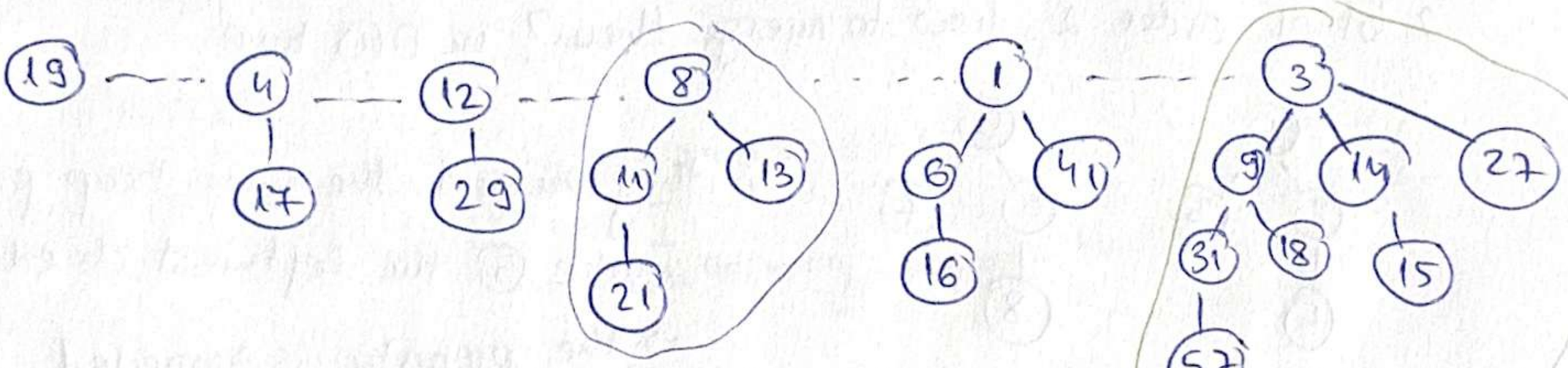
3) Operations

MERGE - two trees ✓

- two heaps

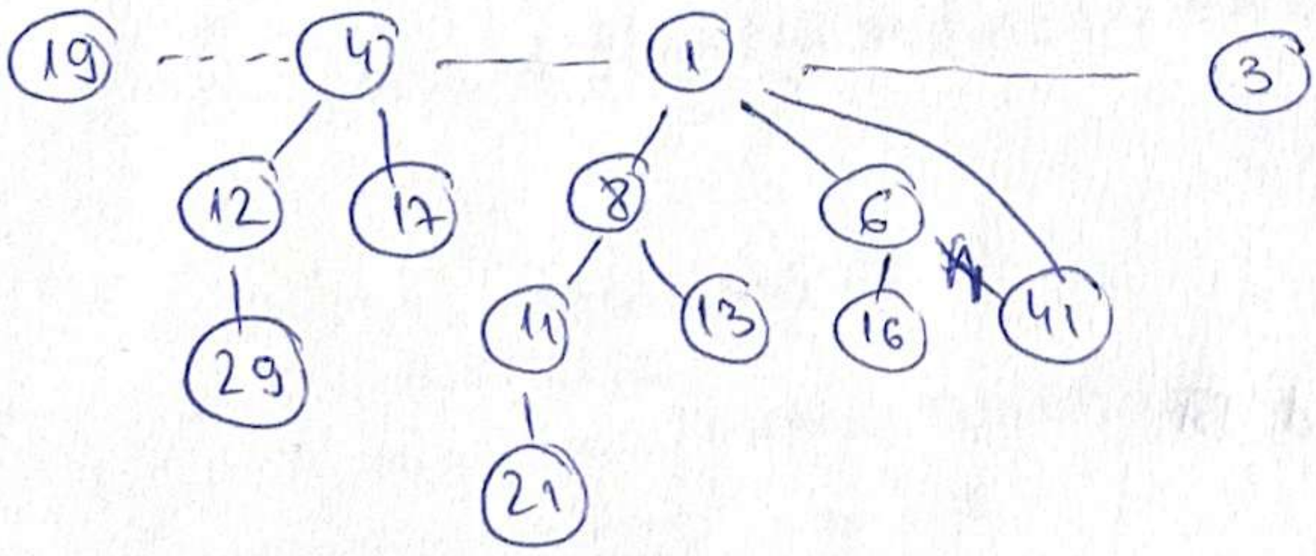


merge sorted LL

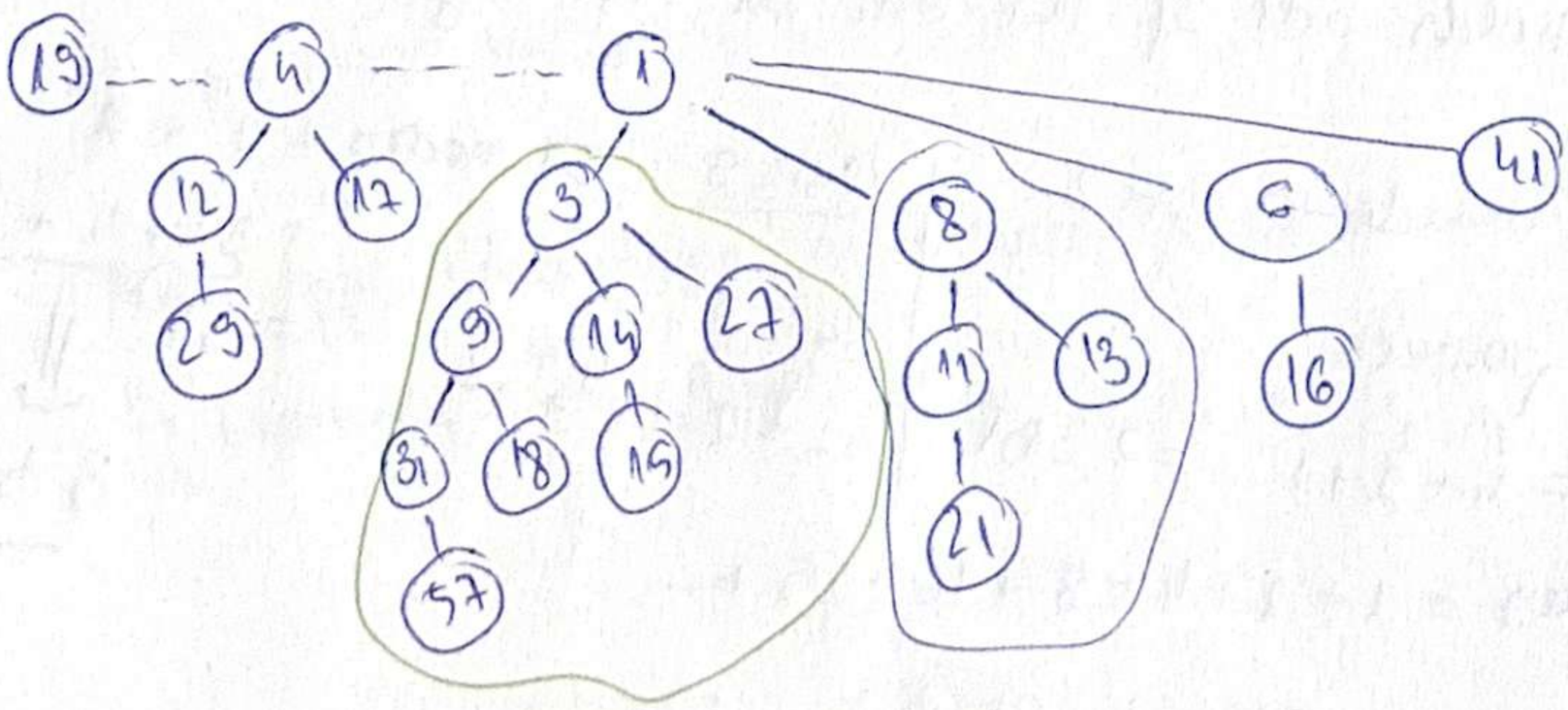


merge ones with same order.

(keeping in mind the property)



...



- Complexity of merging: O (nr of trees (H_1) + nr of trees (H_2))
  - => logarithmic => $o(\log_2 n)$?
  - => better complexity than ....

PUSH

POP

TOP

→ Complexity of adding: „pretty much the same" as merge...?

=> not easier than merging

=> WC: go through everything

→ Complexity of removing.

=> ex: remove the root, ~~ch~~ swap (3) ↔ (41) => valid heap.

=> $o(\log_2 n)$

4) Public transportation service

- => display timetable
- → display info for bus line
- → fast! => array => keep them sorted → use bus nr as index!
- => rename bus lines

=> DATable

simple / efficient

$\Theta(1)$

ADD new line

=> index 72 in DAT

issues:

→ 24B

→ memory

HASH TABLE

based on arrays

two ≠ keys produce the same array index

might not be numbers

collisions (normal)

?

→ function that gives position $\overline{0, m-1}$ => $h(key) \to 0 \dots m-1$

?? T [key] ← value (DAT)

T [h(key)] ← value (HT)

(3)

5) What makes a good ≠ hash function?

→ minimize collisions

→ deterministic

→ $\Theta(1)$ time

→ assumption of simple uniform hashing

- DIVISION METHOD: $h(k) = k \bmod m$ ← in case we want big values (CNP) represented in the array?

m → parameter → hash table size

capacity ≫ prime ≫ nr. not too close to pw or 2.

- MID-SQUARE METHOD:

m → ...

h(4567)

$4567 * 4567 = 20857489 = 57.$

↓

most mixing happens here!

- MULTIPLICATION METHOD:

m → ...

fractional, floor:

$h(k) = \text{floor}(m * \text{frac}(k * A))$ — frac: fractional part

↓

$\in (0,1)$

UNIVERSAL HASHING?

$h(k) = k \bmod m$ HASH FUNCTION most used

$m = 7$

$h(10) = 10 \% 7 = 3$

$h(3) = 3 \% 7 = 3$ } COLLISIONS

$h(17) = 17 \% 7 = 3$

Avoid ppl who take your hash function to cause collisions

↳ "Example 1", "Example 2", 3

6) what if keys are not numbers? How could we define our hash function?

→ hash functions specific for numbers (a lot)

I. Key $\xrightarrow{\text{convert}}$ number

II.

III. multiply every numerical code with ...

$$\underbrace{s[0]}_{\text{from the string}} * 26^{n-1} + \underbrace{s[1]}_{\text{from the string}}$$

7) new password? Criptography

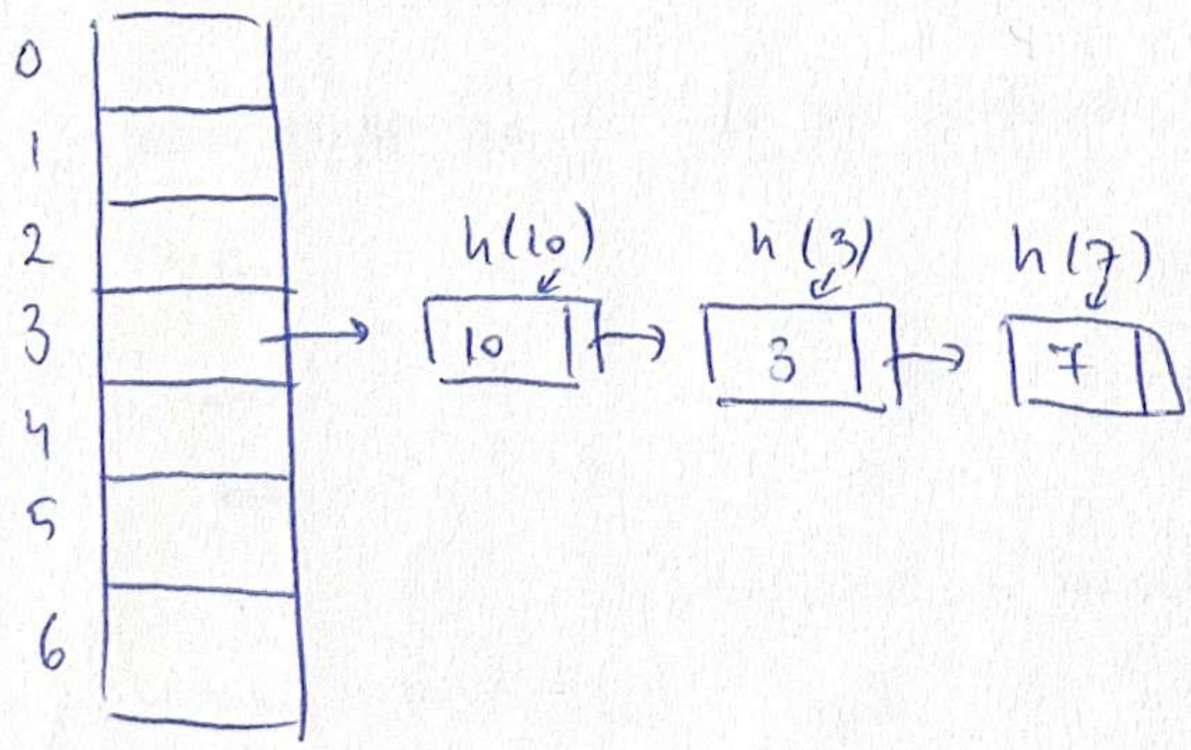
↳ they don't know it → they pass it to the hash function →

→ see if they match

$$\left.\begin{matrix} h(x) = h(y) \\ x \neq y \end{matrix}\right\} \Rightarrow \text{COLLISION.}$$

8) birthdays

- 367 ppl => at least 2 w same bd
- 23 ppl are enough for 50%.



separate chaining ($\alpha > 1$)

we don't want $O(0.5)$ ↓ α

Complexity of searching: $O(\text{length of LL}) \sim O(\alpha + 1)$ ← AVERAGE COMPLEXITY

$\alpha = \frac{n}{m}$ how "full" the hash table is.

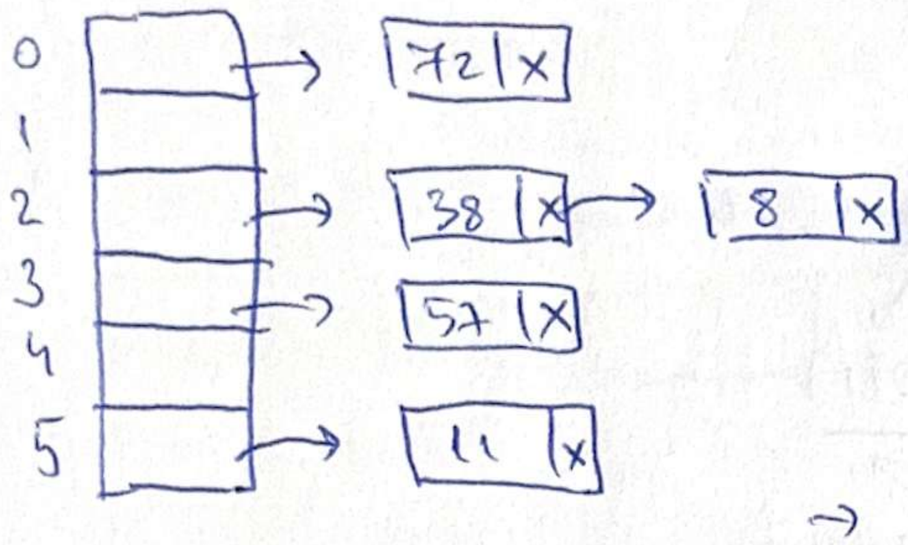
↳ ?

$\frac{3}{7}$ ↳ AVG length of a list

! Complexities on average are constant as long as α is small enough.

Exam: Collision-resolution method:

$m = 6$



→ insert elements in hash table

$h(38) = 38 \% 6 = 2$, $\alpha = \frac{1}{6} < 0.7$.

$h(11) = 11 \% 6 = 5$ → empty pos , $\alpha = \frac{2}{6} < 0.7$

$h(8) = 8 \% 6 = 2$, $\alpha = \frac{3}{6} < 0.7$.

$h(72) = 72 \% 6 = 0$, $\alpha = \frac{4}{6} < 0.7$.

$h(57) = 57 \% 6 = 3$, $\alpha = \frac{5}{6} > 0.7$ STOP

→ resize , rehash

$h(11) = 11 \% 12 = 11$ (search for 11) => not there although it is there (problem!)

↓
new m